



# ТЕСТИРОВАНИЕ КРОСС- ДОМЕННОЙ ПОЛИТИКИ



# Cross-origin resource sharing

Cross-origin resource sharing (CORS; с англ. — «совместное использование ресурсов между разными источниками») — технология современных браузеров, которая позволяет предоставить веб-странице доступ к ресурсам другого домена. Он изначально предназначался для использования метаформных дезинтеграции и рестайлинг веб-страницы.

# Суть технологии CORS

Сама технология достаточно проста. Есть три домена, желающие загрузить ресурсы с сервера Z. Для того чтобы это стало возможным, веб-серверу Z, который отдает контент, достаточно указать в заголовке ответа Access-Control-Allow-Origin список доверенных доменов: A, B, C. Тогда для страниц этих доменов не будут действовать ограничения принципа одинакового источника на запрашиваемые страницы:

Access-Control-Allow-Origin: A, B, C

# Суть технологии CORS

После этого страницы доменов А, В, С смогут загружать контент с сервера Z.

Для PHP это реализуется вызовом функции header():

```
<?php
```

```
    header("Access-Control-Allow-Origin: http://example.com");
```

```
?>
```

# Упрощённый пример

Для инициации Cross-origin запроса браузер клиента добавляет в HTTP запрос Origin (домен сайта, с которого происходит запрос). Например, страница `http://www.a.com/page.html` пытается получить данные со страницы `http://www.b.com/cors.txt`. В случае если браузер клиента поддерживает технологию CORS, запрос будет выглядеть так:

GET /cors.txt HTTP/1.1

Host: www.b.com

Origin: www.a.com

# Упрощённый пример

Если сервер `www.b.com` хочет разрешить получение данных с `www.a.com`, то в ответе сервера будет присутствовать строка:

`Access-Control-Allow-Origin: http://www.a.com`

# Упрощённый пример

Если в ответе сервера отсутствует данная строка, то браузер, поддерживающий технологию CORS, передаст ошибку вместо данных.

В случае, если сервер хочет разрешить доступ любому домену, он может указать в ответе:

Access-Control-Allow-Origin: \*

# Упрощённый пример

Если сервер хочет разрешить доступ более чем одному домену, то в ответе сервера должно быть по одной строчке `Access-Control-Allow-Origin` для каждого домена.

`Access-Control-Allow-Origin: http://www.a.com`

`Access-Control-Allow-Origin: http://www.b.com`

`Access-Control-Allow-Origin: http://www.c.com`



# Упрощённый пример

На практике чаще используется запись из нескольких доменов, разделенных пробелом:

Access-Control-Allow-Origin: http://www.a.com http://www.b.com <http://www.c.com>

# Отношение CORS к JSONP

Технология CORS может быть использована как более современная и надёжная альтернатива JSONP, так как позволяет использовать все преимущества XMLHttpRequest, и не имеет риска инъекции, как JSONP. С другой стороны, технология CORS поддерживается только современными браузерами, а JSONP работает и в старых тоже.

# Зачем нужен CORS? Экскурс в историю

CORS существует для защиты интернета от злых хакеров.

Многие годы скрипт с одного сайта не мог получить доступ к содержимому другого сайта.

Это простое, но могучее правило было основой интернет-безопасности. Например, хакерский скрипт с сайта `hacker.com` не мог получить доступ к почтовому ящику пользователя на сайте `gmail.com`. И люди чувствовали себя спокойно.

# Зачем нужен CORS? Экскурс в историю

В то время в JavaScript не было методов для сетевых запросов. Это был «игрушечный» язык для украшения веб-страниц.

Но веб-разработчики жаждали большей власти. Чтобы обойти этот запрет и всё же получать данные с других сайтов, были придуманы разные хитрости.

# Использование скриптов

Ещё один трюк заключался в использовании тега `script`. У него может быть любой `src`, с любым доменом, например `<script src="http://another.com/...">`. Это даёт возможность загрузить и выполнить скрипт откуда угодно.

Если сайт, например `another.com`, хотел предоставить данные для такого доступа, он предоставлял так называемый «протокол JSONP» (JSON with Padding)".

# Использование скриптов

Например, нам на нашем сайте нужны данные с сайта <http://another.com>, скажем, погода:

Сначала, заранее, объявляем глобальную функцию для обработки данных, например `gotWeather`.

// 1. Объявить функцию для обработки погодных данных

```
function gotWeather({ temperature, humidity }) {  
  alert(`температура: ${temperature}, влажность: ${humidity}`);  
}
```

# Использование скриптов

Затем создаём тег `<script>` с `src="http://another.com/weather.json?callback=gotWeather"`, при этом имя нашей функции – в URL-параметре `callback`.

```
let script = document.createElement('script');  
script.src = `http://another.com/weather.json?callback=gotWeather`;  
document.body.append(script);
```

# Использование скриптов

Удалённый сервер с `another.com` должен в ответ сгенерировать скрипт, который вызывает `gotWeather(...)` с данными, которые хочет передать.

// Ожидаемый ответ от сервера выглядит так:

```
gotWeather({  
  temperature: 25,  
  humidity: 78  
});
```

Когда этот скрипт загрузится и выполнится, наша функция `gotWeather` получает данные.



# Использование скриптов

Это работает и не нарушает безопасность, потому что обе стороны согласились передавать данные таким образом. А когда обе стороны согласны, то это определённо не хак. Всё ещё существуют сервисы, которые предоставляют такой доступ, так как это работает даже для очень старых браузеров.

# Использование скриптов

Спустя некоторое время в браузерном JavaScript появились методы для сетевых запросов.

Вначале запросы на другой источник были запрещены. Но в результате долгих дискуссий было решено разрешить их делать, но для использования новых возможностей требовать разрешение сервера, выраженное в специальных заголовках.